
MetaPilot

An AI-enabled web application for end-to-end genome-scale metabolic modelling in natural language.

A concise reference for MetaPilot, centred on **case studies** of the system in use, with supporting sections on the supported **Python tools** and **backends**.

Project Resources

Project repository — MetaPilot application

https://github.com/NiravBhattLab/Agenti_AI_for_COBRA/tree/MetaPilot

Project repository — analysis & data generation

<https://github.com/saathviksista/MetaPilot-analysis-data-generation>

Full technical report

https://github.com/NiravBhattLab/Agenti_AI_for_COBRA/blob/MetaPilot/Documentation/MetaPilot_Technical_Report.pdf

Procedural Knowledge Base (PKB vector database)

https://huggingface.co/datasets/sistasaaathvik/gsm_procedural_knowledge_base

Contents

1	Case Studies	1
1.1	MetaInteract: Agentic Chat Mode	1
1.1.1	Case Study 1: <i>De Novo</i> Model Reconstruction and Quality Assessment . .	1
1.1.2	Case Study 2: Gene Essentiality Analysis in <i>Helicobacter pylori</i>	4
1.2	MetaPlan: Multi-step Plan Mode	5
1.2.1	Case Study: Breast Cancer Context-Specific Modelling and HK2 Drug Simulation	5
2	Supported Python Tools	8
2.1	Specialised Package Dependencies	8
3	Supported Backends	10

1 Case Studies

All case studies that follow used the `gemini-3.1-pro-preview` model.

1.1 MetaInteract: Agentic Chat Mode

MetaInteract is the agentic chat interface of the system, enabling users to interact with genome-scale metabolic models through free-form natural language. Queries are processed by a ReAct-based LLM agent (Gemini 3.1 Pro Preview) that reasons over the available COBRApy-backed tool suite and invokes the appropriate tool for each request. The following two case studies illustrate representative interactions spanning model reconstruction, quality assessment, and gene essentiality analysis.

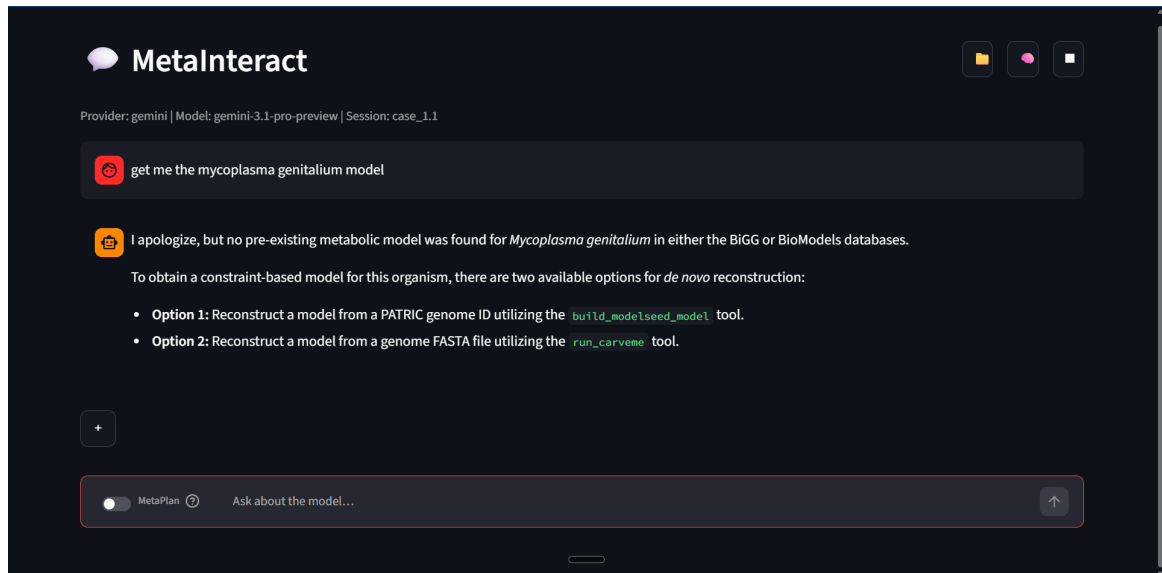
1.1.1 Case Study 1: *De Novo* Model Reconstruction and Quality Assessment

This case study demonstrates the system’s ability to handle a model request that cannot be fulfilled from an existing database (BiGG, BioModels) and to guide the user to an automated *de novo* reconstruction workflow. The interaction is conducted in four turns, each invoking a distinct tool.

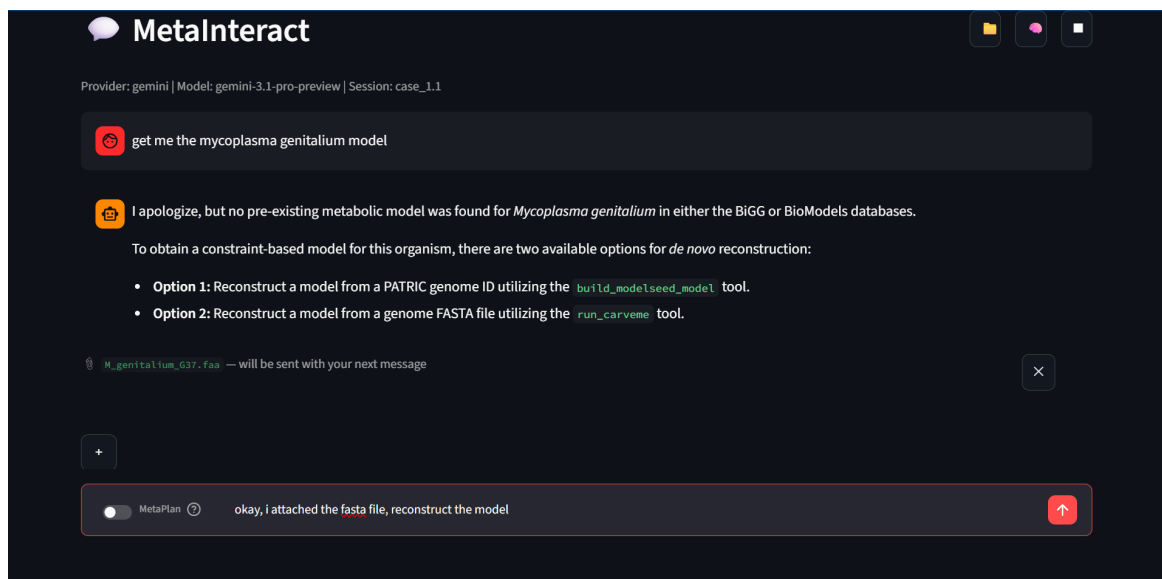
Turn 1 — Model retrieval attempt. The user issues the query “*get me the mycoplasma genitalium model.*” The agent searches the BiGG and BioModels databases but finds no pre-existing model for *Mycoplasma genitalium*. Rather than returning a bare error, it presents two alternative reconstruction pathways: (i) automated draft reconstruction from a PATRIC genome identifier using the `build_modelseed_model` tool, or (ii) homology-based reconstruction from a protein FASTA file using the `run_carveme` tool (Figure 1.1a).

Turn 2 — CarveMe reconstruction. The user attaches the proteome FASTA file `M_genitalium_G37.faa` and requests reconstruction via CarveMe. The agent invokes `run_carveme`, which completes successfully and returns a structured summary of the reconstructed model (Figure 1.1b, 1.2a). The resulting model, assigned the identifier `M_genitalium_G37`, comprises 871 reactions, 653 metabolites, and 130 genes, and is saved as an SBML file at `artifacts/case_1.1/outputs/carveme/M_genitalium_G37.xml`.

Turn 3 — Memote quality assessment. The user then requests “*run memote on the model.*” The agent invokes the `memote_report` tool, which executes the Memote test suite against the reconstructed model and returns a structured evaluation summary (Figure 1.2b). The model achieves an overall Memote score of 87.3% across 74 tests (38 passed, 36 failed, 3 skipped). Despite this score, Memote reports an overall status of *Failed*, which is consistent with the expected behaviour of automated CarveMe reconstructions: Memote’s test suite includes strict annotation-completeness checks that freshly reconstructed models routinely fail in the absence of manual curation. The 87.3% score nonetheless indicates strong metabolic consistency and structural integrity of the model, confirming its suitability for downstream constraint-based analysis.



(a) Turn 1 — model retrieval attempt.



(b) Turn 2 — CarveMe reconstruction (input).

Figure 1.1: Case Study 1 (part 1 of 2): retrieval failure and the start of the *de novo* reconstruction of a *Mycoplasma genitalium* metabolic model.

okay, i attached the fasta file, reconstruct the model [File attached: M_genitalium_G37.faa]

The reconstruction of the metabolic model utilizing the provided FASTA file has been successfully completed. Below is the structured summary of the model reconstruction parameters and quantitative characteristics:

Parameter	Value
Status	Success
Model ID	M_genitalium_G37
Output File	artifacts/case_1.1/outputs/carveme/M_genitalium_G37.xml
Prodigal Used	False
Reactions	871
Metabolites	653
Genes	130

MetaPlan ask about the model...

(a) Turn 2 — CarveMe reconstruction (result).

run memote on the model

The Memote evaluation for the model `M_genitalium_G37` has been executed. The test resulted in an overall status of "failed," with 36 out of 74 tests failing. The model achieved a total Memote score of 87.3%.

Below is the detailed breakdown of the test results, scores by section, and the major failures identified during the evaluation.

Memote Evaluation Summary

Metric	Value
Status	Failed
Total Score	87.3%
Tests Passed	38
Tests Failed	36
Tests Skipped	3

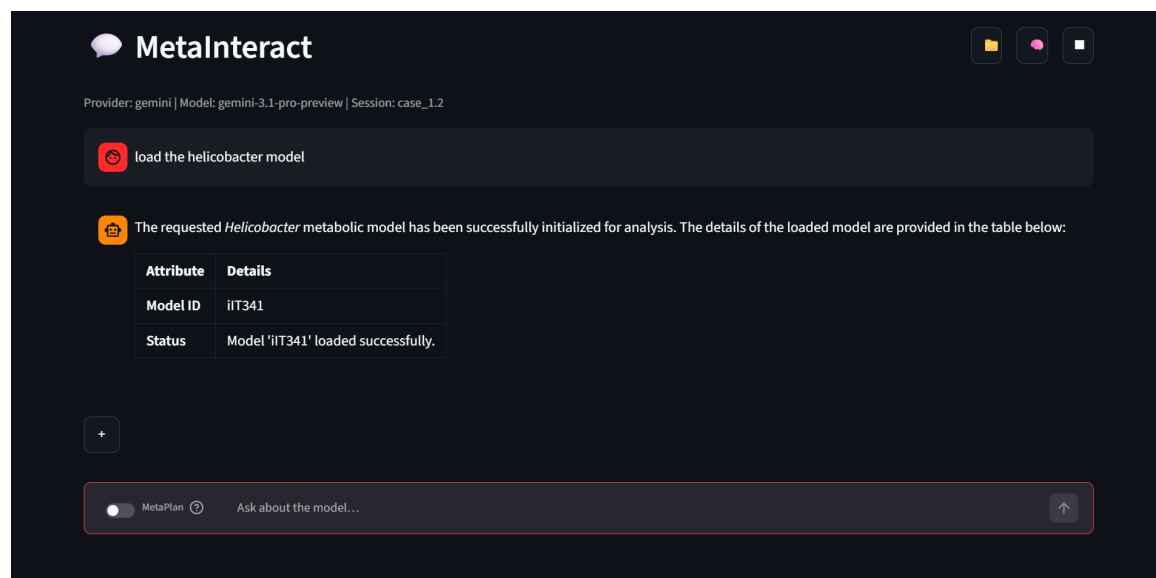
MetaPlan ask about the model...

(b) Turn 3 — Memote quality assessment.

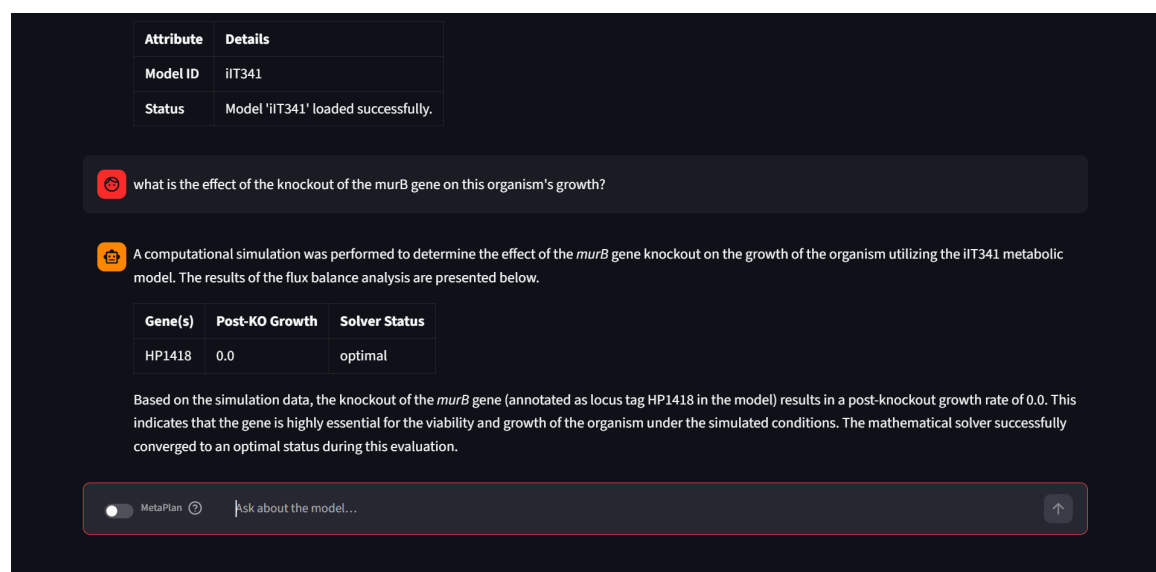
Figure 1.2: Case Study 1 (part 2 of 2): the reconstructed model and its subsequent Memote quality assessment.

1.1.2 Case Study 2: Gene Essentiality Analysis in *Helicobacter pylori*

This case study demonstrates the system’s ability to load a pre-existing curated model and perform a gene knockout simulation from a natural language query, including resolution of a common gene name to its model-internal locus tag identifier.



(a) Turn 1 — model loading.



(b) Turn 2 — gene knockout simulation.

Figure 1.3: Case Study 2 — MetaInteract performs model loading and gene essentiality analysis in the *Helicobacter pylori* iIT341 model.

Turn 1 — Model loading. The user issues the query “load the helicobacter model.” The agent identifies and loads the *Helicobacter pylori* genome-scale metabolic model iIT341¹, confirming

¹I. Thiele, T. D. Vo, N. D. Price, and B. Ø. Palsson. Expanded metabolic reconstruction of *Helicobacter pylori* (iIT341 GSM/GPR): an *in silico* genome-scale characterization of single- and double-deletion mutants. *Journal of Bacteriology*, 187(16):5818–5830, 2005.

successful initialisation (Figure 1.3a).

Turn 2 — Gene knockout simulation. The user then asks “*what is the effect of the knockout of the *murB* gene on this organism’s growth?*” The agent correctly resolves the common gene name *murB* to the model’s internal locus tag HP1418 and invokes `gene_knockout_simulation` (Figure 1.3b). The simulation returns a post-knockout growth rate of 0.0 (solver status: optimal), which the agent interprets as indicating that *murB* is essential for the organism’s viability under the simulated growth conditions. This result is biologically consistent: *murB* encodes UDP-*N*-acetylenolpyruvoylglucosamine reductase, a key enzyme in the peptidoglycan biosynthesis pathway responsible for bacterial cell wall integrity, and its deletion is expected to be lethal in *H. pylori*.

1.2 MetaPlan: Multi-step Plan Mode

1.2.1 Case Study: Breast Cancer Context-Specific Modelling and HK2 Drug Simulation

User Query.

“I want to create a model for a breast cancer cell and then study the growth of the cell after simulating the effects of a drug targeting the gene HK2.”

Overview. This case study exercises the full MetaPlan pipeline: context-specific model reconstruction followed by drug simulation via gene knockout. The GSMPlanner assembled a six-step workflow (Table 1.1, Figure 1.4), drawing from three literature sources, which the executor ran deterministically to completion.

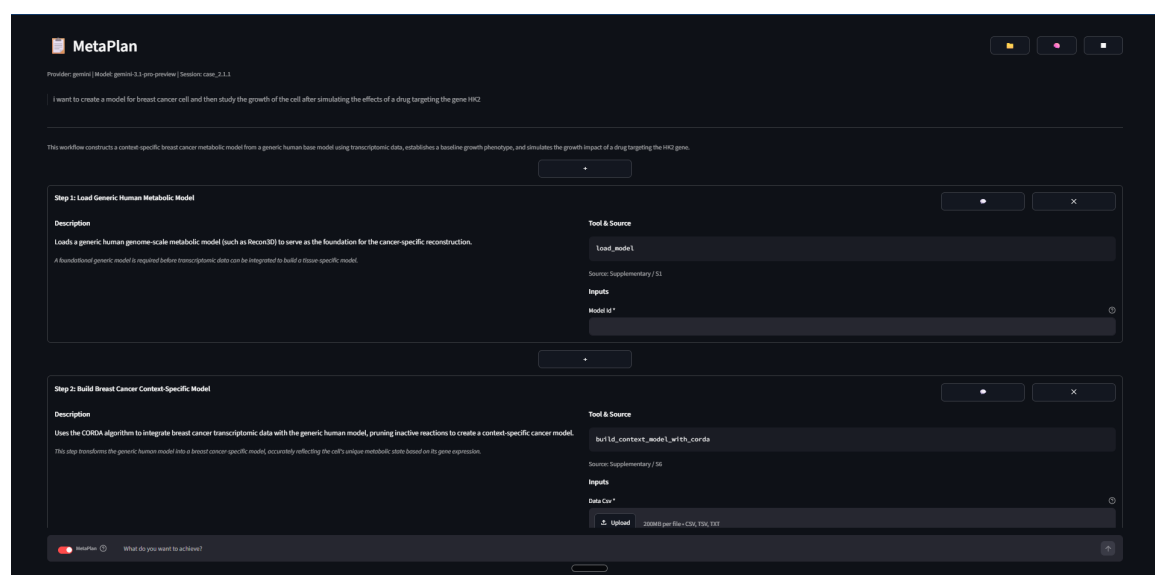


Figure 1.4: MetaPlan interface showing part of the automatically generated six-step plan.

Table 1.1: Six-step plan generated by GSMPlanner for the breast cancer HK2 drug simulation case study, with execution outcomes.

Step	Name	Tool	Status
1	Load Generic Human Metabolic Model	load_model	✓
2	Reconstruct Breast Cancer Model	build_context_model_with_corda	✓
3	Simulate Wild-Type Growth	run_fba	✓
4	Visualise Wild-Type Fluxes	visualize_escher	✓ [†]
5	Simulate HK2 Drug Targeting	gene_knockout_simulation	✓
6	Visualise Knockout Fluxes	visualize_escher	✓

[†] Succeeded after 2 automatic retry attempts.

Execution. Recon3D (10,600 reactions) was loaded and passed to `build_context_model_with_corda` with breast cancer transcriptomic data (`brca_corda_input.csv`). CORDA pruned the reference model to a context-specific network of 4,129 reactions, 1,439 genes, and 2,963 metabolites, with 81.9% of model genes successfully mapped to the expression dataset (Figure 1.5). FBA on the reconstructed model returned an optimal wild-type growth objective of 202.0867. The wild-type flux distribution was visualised on the RECON1.Carbohydrate metabolism Escher map — recovered autonomously after two executor retries — and the *HK2* gene (locus tag 3099_AT1) was then knocked out to simulate drug targeting.

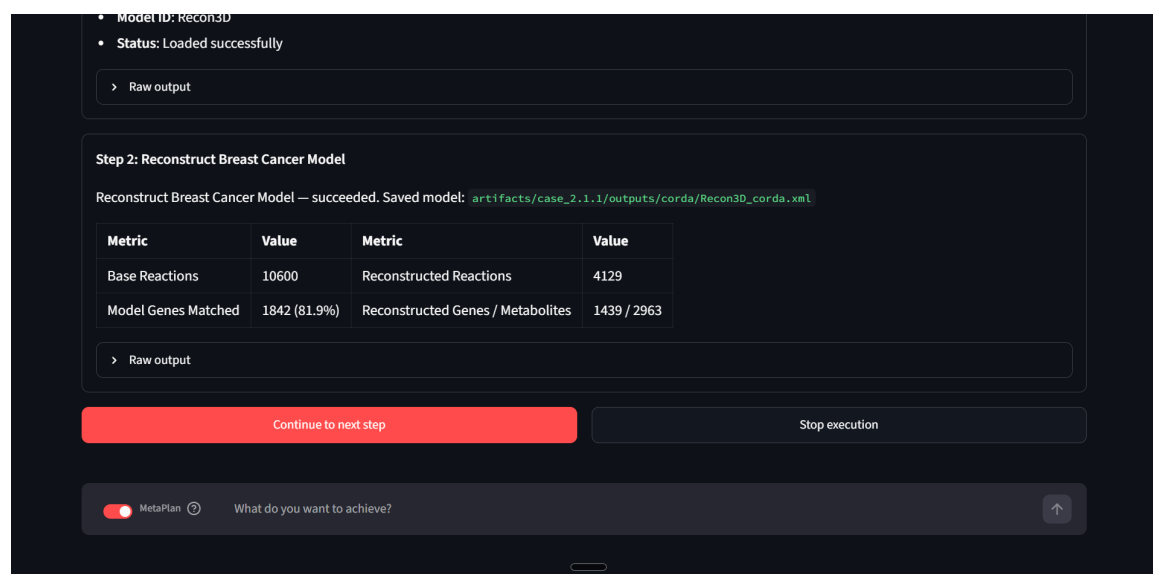


Figure 1.5: Step 2 result: CORDA reconstruction metrics showing the reduction from 10,600 base reactions to a 4,129-reaction breast cancer model.

Result. The post-knockout growth objective remained **202.0867** — identical to the wild-type — indicating that the reconstructed breast cancer model is metabolically robust to single-gene deletion of *HK2*. The Plan Complete screen (Figure 1.6) summarised this as:

Targeting the HK2 gene has no effect on the growth rate of the simulated breast cancer cell. The context-specific breast cancer model was successfully reconstructed from the Recon3D base model, yielding a specialised network of 4,129 reactions. Baseline simulations established an optimal wild-type growth objective value of 202.0867. Following the simulated drug knockout of the HK2 gene (ID 3099_AT1), the cell's growth rate remained entirely unchanged at 202.0867, indicating metabolic redundancy or alternative pathways compensating for the loss.

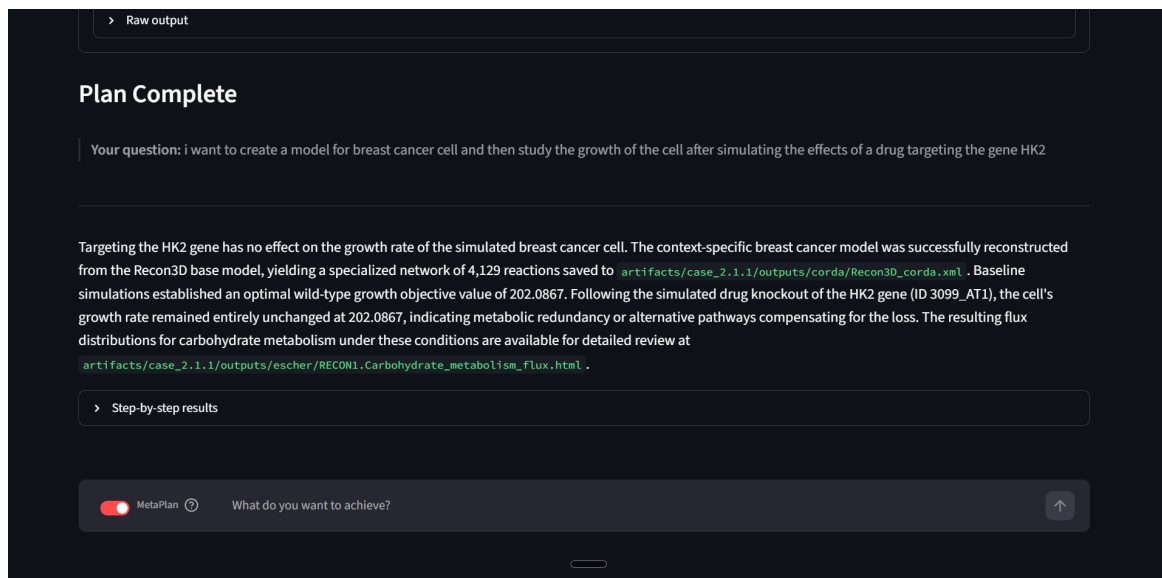


Figure 1.6: Plan Complete screen with the user's query answered, achieving the user's goal.

Discussion. The case study validates three MetaPlan properties. The planner correctly sequenced reconstruction before simulation and sourced steps from multiple papers without manual intervention. The executor demonstrated self-recovery by autonomously retrying a failed Escher map-name resolution. The result itself — no growth change upon *HK2* deletion — is a biologically valid prediction reflecting the metabolic redundancy of alternative hexokinase isozymes retained in the CORDA model, and is communicated directly to the user in the Plan Complete summary.

2 Supported Python Tools

MetaPilot exposes a suite of **38 standalone Python tools** to the agent. Each tool wraps one or more operations and returns a uniform, JSON-serialisable dictionary containing either a structured result payload or an `error` key, so that every tool behaves consistently whether it is invoked by the MetaInteract chat agent or dispatched directly by the MetaPlan executor. The tools are organised below by their phase in the modelling lifecycle.

Table 2.1: MetaPilot Python tool suite, organised by modelling lifecycle phase (38 tools). Tools that rely on a specialised external package are cross-referenced in Table 2.2.

Lifecycle Phase	Tool	Tool	Tool
<i>Model Acquisition & Inspection</i>	<code>load_model</code>	<code>check_model_loaded</code>	<code>get_current_model_id</code>
	<code>model_data</code>	<code>model_info</code>	<code>reaction_info</code>
	<code>metabolite_info</code>	<code>gene_info</code>	
<i>Model Reconstruction</i>	<code>build_model_with_carveme*</code>	<code>build_context_model_with_corda*</code>	<code>build_model_with_mackinac*</code>
<i>Model Curation & Quality Control</i>	<code>run_memote_quality_report*</code>	<code>check_model_consistency</code>	<code>check_mass_balance</code>
	<code>prune_unused_reactions</code>	<code>prune_unused_metabolites</code>	<code>find_blocked_reactions</code>
	<code>run_gap_filling</code>		
<i>Model Configuration & Editing</i>	<code>set_model_objective_value</code>	<code>set_reaction_bounds</code>	<code>add_reaction_to_model</code>
	<code>remove_genes</code>	<code>find_minimal_medium</code>	
<i>Flux Analysis & Simulation</i>	<code>run_flux_balance_analysis</code>	<code>run_parsimonious_fba</code>	<code>run_geometric_fba</code>
	<code>run_flux_variability_analysis</code>	<code>run_production_envelope</code>	<code>sample_metabolic_model</code>
<i>Perturbation & Knockout Analysis</i>	<code>gene_knockout_simulation</code>	<code>reaction_knockout_simulation</code>	<code>run_moma</code>
	<code>run_room</code>	<code>find_essential_genes</code>	<code>find_essential_reactions</code>
<i>Visualization</i>	<code>visualize_escher*</code>	<code>list_escher_maps*</code>	
<i>Utility</i>	<code>request_file_upload</code>		

* Backed by a specialised external package — see Table 2.2. All remaining tools are built directly on COBRApy (<https://github.com/opencobra/cobrapy>).

2.1 Specialised Package Dependencies

The majority of the tool suite is built directly on **COBRApy** (<https://github.com/opencobra/cobrapy>), the de facto standard library for constraint-based modelling in Python. A subset of tools, however, wrap dedicated third-party packages for tasks such as *de novo* and context-specific reconstruction, automated quality assessment, and pathway visualisation. These packages, the tools that depend on them, and links to their project pages are listed in Table 2.2, so that users can consult the upstream documentation directly.

CarveMe performs fast, homology-based *de novo* reconstruction of genome-scale models from a genome or proteome. **CORDA** (Cost Optimization Reaction Dependency Assessment) prunes a generic reference model into a context-specific network using omics evidence. **Mackinac** bridges the ModelSEED service and COBRApy to build draft models from genome identifiers. **Memote** runs a standardised community test suite that scores a model’s structural and annotation quality. **Escher** builds and renders interactive metabolic pathway maps for overlaying flux distributions.

Table 2.2: Tools backed by specialised external packages, with reference links.

Package	Tool(s)	Reference
CarveMe	build_model_with_carveme	https://github.com/cdanielmachado/carveme
CORDA	build_context_model_with_corda	https://github.com/resendislabs/corda
Mackinac (ModelSEED bridge)	build_model_with_mackinac	https://github.com/mmundy42/mackinac
Memote	run_memote_quality_report	https://github.com/opencobra/memote
Escher	visualize_escher list_escher_maps	https://github.com/zakandrewking/escher

3 Supported Backends

MetaPilot is deliberately **model-agnostic**. All large language model (LLM) interactions are routed through a single factory function, `get_llm(provider, model, api_key)`, which returns a `LlamaIndex-compatible LLM` object regardless of the underlying provider. The active provider and model can be switched at runtime without restarting the application, so the same workflows run unchanged across hosted commercial APIs and locally-served open models. This flexibility is especially relevant for deployments where data privacy or fully offline operation is required, in which case a locally-hosted backend can be substituted with no other changes to the system.

The system supports **six backends**: four hosted LLM API providers (Groq, OpenAI, Google Gemini, and Hugging Face Inference) and two local serving runtimes (Ollama and llama.cpp for GGUF models). Table 3.1 lists each backend alongside a reference link where an API key can be created, or the runtime obtained.

Table 3.1: LLM backends supported by MetaPilot. Hosted providers require an API key from the linked console; local runtimes are self-hosted and require no key.

Backend	Type	Reference / API access
Groq	Hosted API	https://console.groq.com/keys
OpenAI	Hosted API	https://platform.openai.com/api-keys
Google Gemini	Hosted API	https://aistudio.google.com/app/apikey
Hugging Face Inference	Hosted API	https://huggingface.co/settings/tokens
Ollama	Local runtime	https://ollama.com
llama.cpp	Local runtime	https://github.com/ggml-org/llama.cpp

The four hosted providers—**Groq**, **OpenAI**, **Google Gemini**, and **Hugging Face Inference**—serve models over the network and require an API key generated from the linked console. The two local runtimes let the system run entirely on-premises: **Ollama** provides a convenient local server for a curated set of open models, while **llama.cpp** serves quantised GGUF models directly, including on commodity hardware. Because all six are reached through the same `get_llm` factory, switching between a cloud API and a local model is a configuration change rather than a code change.